

23L_6006 Mudassar Hussain
6B2

Lab 5

Sample code

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
int main(int argc, char *argv[])
{
    int pfd[2];
    char buf;
    pipe(pfd);
    if (fork() == 0)
    {
        /* Child reads */
        close(pfd[1]);
        while (read(pfd[0], &buf, 1) > 0)
            printf("%c", buf);
        close(pfd[0]);
    }
    else
    {
        /* Parent writes */
        close(pfd[0]);
        write(pfd[1], argv[1], strlen(argv[1]));
        close(pfd[1]);
        wait(NULL);
    }
    return 0;
}
```

```
ubuntu@ubuntu24-04: ~/Desktop/lab5
ubuntu@ubuntu24-04:~/Desktop/lab5$ gcc sample.c -o sample
ubuntu@ubuntu24-04:~/Desktop/lab5$ ./sample
Segmentation fault (core dumped)
ubuntu@ubuntu24-04:~/Desktop/lab5$ ./sample 2
2ubuntu@ubuntu24-04:~/Desktop/lab5$
```

Q1

```
#include<stdio.h>
#include <unistd.h>
#include <cstring>
```

```
#include <sys/wait.h>
```

```
using namespace std;
```

```
int main(int argc, char *argv[]) {
    int fd[2];
    const char*message = "Hello";
    char buf;
```

```
    pipe(fd);
```

```
    //Child Process
```

```
    if (fork() == 0) {
        close(fd[1]);
```

```
        printf("%s\n",message);
        close(fd[0]);
```

```
    }
```

```
    // Parent Process
```

```
    else {
        close(fd[0]); //close unused read end
```

```

    write(fd[1], message, sizeof(buf));
    close(fd[1]); // Close write end
    wait(NULL);
}

return 0;
}

```



A terminal window titled 'ubuntu@ubuntu24-04: ~/Desktop/lab5' showing the compilation and execution of a C++ program. The user runs 'g++ q1.cpp -o q1' three times, followed by './q1 Hello' three times. The output shows 'Hello' being printed each time.

```

ubuntu@ubuntu24-04:~/Desktop/lab5$ g++ q1.cpp -o q1
^[[Aubuntu@ubuntu24-04:~/Desktop/lab5$ ./q1 Hello

ubuntu@ubuntu24-04:~/Desktop/lab5$ g++ q1.cpp -o q1
ubuntu@ubuntu24-04:~/Desktop/lab5$ ./q1 Hello

ubuntu@ubuntu24-04:~/Desktop/lab5$ g++ q1.cpp -o q1
ubuntu@ubuntu24-04:~/Desktop/lab5$ ./q1 Hello
Hello
ubuntu@ubuntu24-04:~/Desktop/lab5$ 

```

Q2

```

#include <iostream>
#include <unistd.h>
#include <sys/wait.h>

```

```
using namespace std;
```

```

int main() {
    int pfd[2];

    // Create pipe
    if (pipe(pfd) == -1) {
        perror("Pipe failed");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
}

```

```

}

// Child Process, Consumer
if (pid == 0) {
    close(pfd[1]); // Close unused write end

    int number;

    cout << "Consumer received numbers: ";

    // Read numbers from pipe
    while (read(pfd[0], &number, sizeof(number)) > 0) {
        cout << number << " ";
    }

    cout << endl;
    close(pfd[0]); // Close read end
}

//Parent Process , Producer
else {
    close(pfd[0]);

    // Generate numbers 1 to 10
    for (int i = 1; i <= 10; i++) {
        write(pfd[1], &i, sizeof(i));
    }

    close(pfd[1]); // Close write end
    wait(NULL); // Wait for child
}

return 0;
}

```

```
ubuntu@ubuntu24-04:~/Desktop/Lab5$ g++ q1.cpp -o q1
ubuntu@ubuntu24-04:~/Desktop/Lab5$ ./q1
Consumer received numbers: 1 2 3 4 5 6 7 8 9 10
ubuntu@ubuntu24-04:~/Desktop/Lab5$
```

Q3

```
#include<iostream>
#include <unistd.h> // for pipe(), fork(), read(), write()
#include <cstring>
#include <cctype>
#include <sys/wait.h> // for wait()
```

```
using namespace std;
```

```
int main() {
    int pipe1[2]; // pipe1: parent -> child
    int pipe2[2]; // pipe2: child -> parent

    // Create the pipes
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        cerr << "Pipe creation failed!" << endl;
        return 1;
    }
}
```

```
pid_t pid = fork(); // create child process
```

```
if (pid < 0) {
    cerr << "Fork failed!" << endl;
    return 1;
}
else if (pid > 0) {
```

```

// Parent
close(pipe1[0]); // Close read end of pipe1
close(pipe2[1]); // Close write end of pipe2

string message;
cout << "Enter a message: ";
getline(cin, message);

// Send message to child
write(pipe1[1], message.c_str(), message.length() + 1);
close(pipe1[1]); // Close write end after sending

// Read modified message from child
char buffer[1024];
read(pipe2[0], buffer, sizeof(buffer));
cout << "Modified message from child: " << buffer << endl;
close(pipe2[0]);

wait(NULL); // Wait for child to finish
}
else {
    // Child Process
    close(pipe1[1]); // Close write end of pipe1
    close(pipe2[0]); // Close read end of pipe2

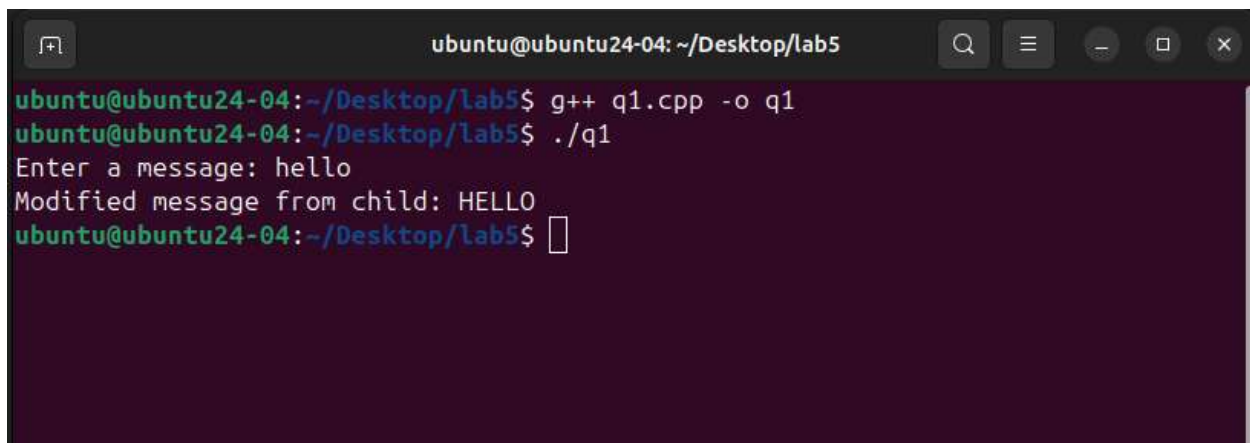
    // Read message from parent
    char buffer[1024];
    read(pipe1[0], buffer, sizeof(buffer));
    close(pipe1[0]);

    // Reverse case of each character
    for (int i = 0; buffer[i] != '\0'; i++) {
        if (islower(buffer[i]))
            buffer[i] = toupper(buffer[i]);
        else if (isupper(buffer[i]))
            buffer[i] = tolower(buffer[i]);
    }

    // Send modified message back to parent
    write(pipe2[1], buffer, strlen(buffer) + 1);
}

```

```
        close(pipe2[1]);  
        exit(0);  
    }  
  
    return 0;  
}
```



A terminal window titled 'ubuntu@ubuntu24-04: ~/Desktop/lab5' with standard window controls. The terminal shows the compilation and execution of a C++ program. The user enters 'hello' at the prompt, and the program outputs 'Modified message from child: HELLO'.

```
ubuntu@ubuntu24-04:~/Desktop/lab5$ g++ q1.cpp -o q1  
ubuntu@ubuntu24-04:~/Desktop/lab5$ ./q1  
Enter a message: hello  
Modified message from child: HELLO  
ubuntu@ubuntu24-04:~/Desktop/lab5$
```